

PENTERACT DORN

DIRECTIVE IMPERIALE V3 — DOCUMENT DE REFERENCE SCELLE
MATRICE PARALLELE // POP-SCIENCE CULTURE ALGORITHMIQUE
AFFILIATION : VIIe LEGION // INFRASTRUCTURE BUDGET : 0.00 EUR
DATE DE SCELLEMENT : 2026-05-29

FREGATE	NOM	STATUT	ROLE	RESUME
F01	POLUX	SCELLEE	Oracle & Curation	Metaprompt Gemini (4 inputs) → plan_de_vol.json + EXIF strip PNG
F02	CASTELLAN	SCELLEE	HUD Controle & Sim.	Streamlit + Canvas JS → simulation + fige le JSON (slider en secondes)
F03	SIGISMUND	SCELLEE	Reacteur Multi-Formes	Remotion — 3 sous-etapes (A/B/C) — timing dynamique
F04A	INWIT	SCELLEE	Viewer + Speed Control	Player HTML Colab + selecteur vitesse 0.5x → 2x
F04B	INWIT	SCELLEE	FFmpeg Finishing	Re-encode + setpts + wipe fingerprint → youtube_*.mp4

// FIN DE TRANSMISSION PRELIMINAIRE // IMPERATOR SOMNIUM //

DELTA V2 → V3 — MODIFICATIONS SCHELLES

- **Timing dynamique** : la duree de la video n'est plus fixe — la geometrie mathematique dicte le nombre de frames via `calculateMetadata` Remotion.
- **Nouveau bloc JSON *timing*** : `fps`, `target_duration_seconds`, `x_range`, `step_per_frame`, `complexity_coefficient`, `final_freeze_frames`, `playback_speed`.
- **animation_speed par courbe** : `reveal_style` (`linear`/`ease_in`/`ease_out`/`dramatic`) + `pace_factor` dans chaque `reactor_curves[n]`.
- **complexity_coefficient par engine_type** : Gemini choisit automatiquement selon le type de visualisation.
- **META_POLUX — 4 inputs** : ajout de la duree cible comme 4e question operateur. Gemini calcule `step_per_frame`.
- **F04 INWIT scinde en F04A + F04B** : F04A = viewer HTML avec selecteur de vitesse ; F04B = FFmpeg finishing conditionnel.
- **F02 CASTELLAN** : slider affiche la progression mathematique (`x = 0 → x_end`) et la duree en secondes, plus uniquement les frames.
- **F03 SIGISMUND** : `MathInterpreter.jsx` lit `timing.phases` et `pace_factor` pour cadencer la revelation SVG `stroke-dashoffset`.

1. GENESE ET PHILOSOPHIE — LE NOUVEL ELEMENT

1.1 — Rupture architecturale

L'analyse des formats academiques classiques de visualisation scientifique revele une barriere invisible : la froideur conceptuelle. Un graphique mathematique pur, aussi parfait soit-il, ne genere pas d'adherence emotionnelle immediate sur les plateformes de format court (Shorts / Reels / TikTok). L'idee fondamentale du Penteract DORN est nee d'une rupture architecturale de type Iron Man 2 : remplacer le combustible traditionnel (les mathematiques abstraites) par un Nouvel Element a haute densite energetique : la Pop-Science Culture.

1.2 — La Regle d'Or de l'Engagement

S'approprier les grands debats iconiques de la pop-culture, du sport de haut niveau et du developpement personnel — l'impact biometrique de Messi, les trajectoires de carriere de Kobe Bryant ou Michael Jordan, la geometrie sacree du ballon Jabulani 2010, la voix de Sia analysee par ondes harmoniques — et les trancher de maniere irrefutable par la geometrie, la physique et le code neon.

L'emotion attire le spectateur. La rigueur mathematique le retient.

1.3 — Le Framework de Reflexion ATOM-IC

Toute decision technique dans DORN est filtree par le meta-framework ATOM-IC, fusion de quatre methodes de resolution de problemes. Ce framework s'applique a l'interieur de chaque fregate — il ne remet jamais en cause les axiomes structurels (fregates, CMS JSON, Tesseract, Penteract).

PHASE	FRAMEWORK SOURCE	QUESTION DE REFERENCE
[A] Analyse Subatomique	First Principles (Elon Musk)	Quel est l'atome de donnee minimal de cette fregate ?
[T] Transmutation Inventive	TRIZ (Altshuller)	Quelle contradiction peut etre retournee en avantage ?
[O] Optimisation Cinetique	Loi de Pareto (95/5)	Quels 5% d'effort produisent 95% de l'impact visuel ?
[M] Manifestation N.U.K.E.	N.U.K.E (Nuclear Unit Kinetic Eng.)	Comment s'execute en 1 commande headless, sans clic ?

N.U.K.E se decompose en 4 sous-principes : Nucleus (atome de donnee), Unconventional coupling (librairies detournees), Kinetic efficiency (CLI headless Colab), Exponential assembly (code qui genere du code pour la fregate suivante).

2. DEFINITIONS GEOMETRIQUES DE L'INFRASTRUCTURE

Le systeme repose sur une topologie logicielle stricte inspiree du stoicisme de Rogal Dorn : aucune fioriture, une structure indestructible.

Le Tesseract (4D)

Unite quadridimensionnelle de livraison finale. Represente un Short unique, stabilise et encapsule. Comprend la video rendue a 60 FPS (9:16 ou 16:9), et les metadonnees SEO.

Le Penteract (5D)

Matrice d'infrastructure a 5 dimensions. Ecosysteme de production isole contenant ses propres fregates, ses propres assets et ses propres regles de calcul. Fonctionne en parallele absolu.

Le CMS JSON (Disque Central)

Coeur logique et immuable. La matiere premiere est textuelle et geometrique. Le *plan_de_vol.json* est le plan de vol universel : il contient les equations brutes, les pointeurs PNG, la config camera, le bloc timing et le format cible.

3. META-FRAMEWORK ATOM-IC — REFERENCE COMPLETE

ATOM-IC (Atomic Transmutation & Optimized Methods for Industrial Creation) est le framework de reflexion utilise pour concevoir chaque brique technique de DORN. Il force a sortir de la methode logicielle traditionnelle pour entrer dans une methode de calcul pur.

[A] — ANALYSE SUBATOMIQUE

Framework source : First Principles

Methode : Avant de coder, deconstruire le probleme jusqu'a ses verites fondamentales. Oublier 'comment font les autres' et regarder la donnee pure.

Exemple DORN : Une animation n'est pas un fichier .fbx. C'est une liste de coordonnees (x,y,z) dans un fichier texte. Si on a l'atome, on le manipule avec un script Python gratuit.

[T] — TRANSMUTATION INVENTIVE

Framework source : TRIZ — Theorie de Resolution des Problemes Inventifs

Methode : Utiliser le genie sovietique pour resoudre des contradictions impossibles (ex: Haute Qualite VS Zero Ressource). Principe d'Inversion ou de Segmentation.

Exemple DORN : Au lieu de faire ramer Colab avec un rendu 4K, rendre en 480p puis transmettre via une IA d'upscaling (Real-ESRGAN). Separer la creation de la qualite.

[O] — OPTIMISATION CINETIQUE

Framework source : Loi de Pareto (regle du 95/5)

Methode : Maximiser l'impact visuel avec le minimum de calcul. Ne pas chercher la perfection technique, chercher l'illusion parfaite.

Exemple DORN : Dans une video virale, l'oeil est attire par la lumiere neon et le mouvement du PNG, pas par la precision mathematique subpixel. On 'nuke' la complexite au profit des effets visuels a fort impact.

[M] — MANIFESTATION N.U.K.E.

Framework source : N.U.K.E (Nuclear Unit for Kinetic Engineering)

Methode : Rendre la solution autonome, headless, CLI-first. Zero clic manuel. N = Nucleus (atome de donnee), U = Unconventional (bibliothèques détournées), K = Kinetic (CLI Colab), E = Exponential (code genre du code).

Exemple DORN : Le script de F01 genere dynamiquement le JSON que F03 va executer. Une commande = une sortie deterministe.

4. SCHEMA DU CMS JSON UNIVERSEL — plan_de_vol.json [v3 MAJ]

Structure de donnees absolue traversant toutes les fregates de la matrice DORN. Produit par META_POLUX (Gemini), enrichi par META_CAMERA (Gemini), valide et fige par F02. Ce fichier est l'unique source de verite du pipeline.

Schema complet avec bloc timing V3 : [v3 NOUVEAU]

```
{
  "matrix_signature": "PENTERACT_DORN_VII_LEGION",
  "concept_metadata": {
    "title": "string — titre viral",
    "hook": "string — accroche 1 phrase",
    "thesis": "string — la these mathematique",
    "engine_type": "time_evolution_comparison | geometric_construction | wave_analysis | single_proof",
    "format": "vertical | horizontal",
    "fps": 60
  },
  "timing": {
    // V3 — bloc timing dynamique
    "fps": 60,
    "target_duration_seconds": 30, // defini par operateur via META_POLUX
    "x_range": { "start": 0, "end": 20 },
    "step_per_frame": 0.05, // calcule par Gemini : x_range / (duration*fps - freeze)
    "complexity_coefficient": 1.0, // selon engine_type (voir table 4.2)
    "final_freeze_frames": 180,
    "playback_speed": 1.0 // editable en F04A (0.5x a 2x)
  },
  "space_environment": {
    "geometry_mode": "cartesian | polar",
    "x_label": "string",
    "y_label": "string",
    "background_asset": "filename.png (optionnel)"
  },
  "reactor_curves": [
    {
      "id": "string",
      "math_input": "string — expression math.js",
      "render_style": {
        "color_hex": "#XXXXXX",
        "glow_radius_px": 20,
        "line_width_px": 6
      },
      "animation_speed": {
        // V3 — vitesse par courbe
        "reveal_style": "linear | ease_in | ease_out | dramatic",
        "pace_factor": 1.0
      },
      "tracking_target": {
        "asset_filename": "face.png",
        "scale_factor": 1.2,
        "physics": {
          "auto_rotate_slope": true,
          "inertia_smooth": 0.1
        }
      }
    }
  ]
}
```

```
    }
  }
],
"camera_plan": {
  "camera_signature": "DORN_META_CAMERA_V1",
  "global_style": {
    "movement_energy": "calm | cinematic | aggressive | viral_edit",
    "default_easing": "easeInOutCubic",
    "shake_intensity": 0.08
  },
  "camera_segments": [
    { "start_frame": 0, "end_frame": 180, "mode": "follow_asset",
      "target_curve_id": "string", "zoom": 1.15, "x_offset": 0,
      "y_offset": -40, "shake": 0.03 }
  ]
},
"final_frame": {
  "annotation": "string — texte de conclusion mathematique",
  "freeze_duration_frames": 90
},
"audio_synthesizer": {
  "wave_type": "sine | square | sawtooth",
  "base_frequency_hz": 220,
  "frequency_multiplier": 12.0
}
}
```

4.1 — Logique du timing dynamique [V3 NOUVEAU]

La duree n'est plus une valeur arbitraire. La geometrie mathematique dicte le nombre de frames. **calculateMetadata** Remotion lit le JSON et compile exactement ce nombre de frames — coupe parfaite, zero manipulation.

```
reveal_frames = (x_range.end - x_range.start) / step_per_frame
total_frames  = reveal_frames * complexity_coefficient + final_freeze_frames

// Calcul inverse (META_POLUX) — Gemini fixe step_per_frame depuis la duree cible :
step_per_frame = x_range / (target_duration_seconds * fps - final_freeze_frames)
```

4.2 — complexity_coefficient par engine_type (Gemini choisit) [V3 NOUVEAU]

engine_type	coefficient	Raison
time_evolution_comparison	1.0	Rythme narratif direct — la comparaison se lit lineairement
wave_analysis	0.6	L'oeil saisit vite les ondes — accelrer renforce l'impact
geometric_construction	1.8	La spirale / construction merite d'etre appreciee lentement
single_proof	1.2	Suspense progressif avant la revelation finale

4.3 — Les 4 Engine Types

engine_type	Exemple d'usage	geometry_mode	math_input exemple
time_evolution_comparison	Messi vs CR7 / Kobe vs Iverson	cartesian	1 / (1 + exp(x - 0.87))
geometric_construction	Jabulani + nombre d'or	polar	phi^(2*theta/pi)
wave_analysis	Voix de Sia — ondes harmoniques	cartesian	1.8*sin(440*x) + 0.6*sin(880*x)
single_proof	Une seule equation decisive	cartesian ou polar	1.025^x

4.4 — Note wave_analysis : normalisation x_range [v3 NOUVEAU]

Pour wave_analysis, le x_range est en radians ou en cycles, pas en unites physiques. Gemini doit normaliser dans META_POLUX (ex: 0 → 4*pi pour 2 cycles complets d'une onde).

4.5 — Les modes camera V1

mode	Usage
static	Camera fixe, aucun mouvement
follow_asset	Suit le PNG attache a une courbe
follow_curve_tip	Suit l'extremite mathematique d'une courbe
wide_reveal	Zoom arriere pour reveler toute la construction
push_in	Zoom progressif vers la preuve finale
final_proof_lock	Derniere frame figee, propre, lisible

5. PLAN DE VOL DES FREGATES — TOUTES SCHELLEES

F01 — FREGATE POLUX [ORACLE & CURATION] [V3 MAJ]

STATUT : SCHELLEE — Transforme un concept pop-culture en CMS JSON exploitable. Securite : nettoyage destructif EXIF sur tous les PNG injectes.

META_POLUX — Metaprompt Gemini (hors pipeline, chat manuel) [V3 MAJ]

INPUT OPERATEUR (4 questions V3)	EXEMPLE
1. Sujet + these	Messi est meilleur que CR7 grace a son centre de gravite bas
2. Reponse mathematique	Montrer que stabilite = f(hauteur centre de gravite)
3. Assets PNG disponibles	messi_head.png, cr7_head.png
4. Duree cible [V3 NOUVEAU]	"30 secondes" / "45 secondes" / "1 minute"

Logique interne Gemini (automatique, invisible operateur) :

- Selection automatique du engine_type selon le sujet
- Extraction des equations mathematiques au format math.js
- Generation du hook / titre viral
- Calcul automatique de step_per_frame depuis la duree cible [V3]
- Calcul du complexity_coefficient selon engine_type [V3]
- Redaction de la description de la final_frame
- Selection du format (vertical 9:16 / horizontal 16:9)

Outputs Gemini (6 elements V3) :

#	ELEMENT	EXEMPLE
1	Hook / titre viral	La Physique Prouve Que Messi Est Imbattable
2	Equation principale	$stabilite(h) = 1 / (1 + \exp(h - 0.87))$
3	Description narrative	Courbe Messi au-dessus de CR7 a partir de x=4
4	Preview finale (texte)	Frame finale : ecart Delta = 23% d'avantage
5	Bloc timing calcule [V3]	step_per_frame=0.037, complexity_coefficient=1.0
6	plan_de_vol.json complet	Pret a déposer dans F01/IN/

F02 — FREGATE CASTELLAN [HUD DE CONTROLE & SIMULATION] [V3 MAJ]

STATUT : SCHELLEE — HUD de validation pure. L'operateur voit l'animation simulee en temps reel, corrige si besoin, puis fige le JSON pour F03. Stack : Streamlit + Canvas JS + math.js CDN.

Zone simulation Canvas JS :

- Courbes dessinees en temps reel via math.js (CDN)
- Slider temporel : affiche $x = 0 \rightarrow x_range.end$ ET la duree en secondes [V3]
- Re-dessin instantane a chaque modification d'equation
- Preview final_frame en fin de slider
- Supporte les 4 engine_types

Elements editables dans l'interface (V1) : [V3 MAJ]

ELEMENT EDITABLE	CHAMP JSON MODIFIE
Equation de chaque courbe	reactor_curves[n].math_input
Ajouter une courbe (bouton +)	Ajoute un objet dans reactor_curves
PNG asset par courbe	reactor_curves[n].tracking_target.asset_filename
reveal_style par courbe [V3]	reactor_curves[n].animation_speed.reveal_style
pace_factor par courbe [V3]	reactor_curves[n].animation_speed.pace_factor
Label axe X	space_environment.x_label
Label axe Y	space_environment.y_label

ELEMENT EDITABLE	CHAMP JSON MODIFIE
Titre principal	concept_metadata.title
Phrase d'equation (bas d'ecran)	concept_metadata.thesis
Couleur de chaque courbe	reactor_curves[n].render_style.color_hex
Annotation final_frame	final_frame.annotation
Format (vertical / horizontal)	concept_metadata.format

Bouton final : « **FIGER LE PLAN DE VOL** » → validated_by_magos: true + timestamp ISO → OUT/

F03 — FREGATE SIGISMUND [REACTEUR MULTI-FORMES] [V3 MAJ]

STATUT : SCELLEE — Moteur universel Remotion (React / TypeScript). Le Reacteur Multi-Formes est un moule adaptatif agnostique divise en 4 chambres de combustion. Execution en 3 sous-etapes sur Colab + Modal serverless.

Les 4 Chambres du Reacteur Multi-Formes : [V3 MAJ]

CHAM BRE	NOM	FICHER JSX	ROLE
1	Interpreteur	MathInterpreter.jsx	math.js evalue math_input frame par frame — lit timing.phases + pace_factor [V3]
2	Traceur	CurveTracer.jsx	SVG stroke-dashoffset, revelation progressive selon reveal_style [V3], glow neon
3	Ancrage Assets	AssetTracker.jsx	PNG a l'extremite courante, rotation = arctan(derivee numerique)
4	Camera Virtuelle	VirtualCamera.jsx	Lit camera_plan, applique CSS transform global par segment
-	Assembleur	Composition.jsx	Assemble les 4 chambres, gere les 4 engine_types, lit timing pour calculateMetadata

Logique calculateMetadata V3 : [V3 NOUVEAU]

```
// Composition.jsx - calculateMetadata
const reveal = (x_range.end - x_range.start) / timing.step_per_frame;
const total = Math.ceil(reveal * timing.complexity_coefficient) + timing.final_freeze_frames;
return { durationInFrames: total, fps: timing.fps };
```

Technologie TRIZ appliquee — stroke-dashoffset

Contradiction : 60 FPS fluide sur Modal avec compute limite. Retournement : la courbe complete est pre-dessinee en SVG des la frame 0. On ne calcule pas — on revele frame par frame en changeant un seul parametre CSS. Compute quasi nul par frame. Le reveal_style (linear/ease_in/ease_out/dramatic) applique une fonction d'easing sur le ratio stroke-dashoffset.

Les 3 sous-etapes d'execution :

SOUS-ETAPE	ROLE	SORTIE
F03A — BUILD & TEST	Build Remotion + rendu test 30 frames → validation visuelle	30 frames test + rapport
F03B — DISPATCH MODAL	Decoupe en chunks → lance 3 workers Modal	N chunks en parallele
F03C — RECOVER & MERGE	Recupere chunks → verifie → FFmpeg merge	video_render.mp4

F04 — FREGATE INWIT [LA FORGE CLOUD — FINISHING] [V3 MAJ]

STATUT : SCELLEE — Terminus du pipeline. Scindee en 2 sous-etapes en V3. F04A permet a l'operateur de visionner et de choisir la vitesse finale. F04B applique le finishing FFmpeg complet.

F04A — VIEWER + SPEED CONTROL [V3 NOUVEAU]

Player HTML / IPython dans Colab qui affiche video_render.mp4. Selecteur de vitesse : **0.5x / 0.75x / 1x / 1.25x / 1.5x / 2x**. L'operateur valide la vitesse → F04B recoit le parametre *playback_speed*.

F04B — FINISHING FFMPEG [V3 MAJ]

ET AP E	OPERATION	DETAIL
1	QA pre	Duree coherente, stream video present, taille > 100 KB

ET AP E	OPERATION	DETAIL
2	Detection format	Lit concept_metadata.format → vertical = youtube_short.mp4 / horizontal = youtube_long.mp4
3	Speed filter [V3]	Si playback_speed != 1.0 → FFmpeg setpts=PTS/speed. Sinon : saute le filtre (pas de re-encode inutile)
4	FFmpeg camouflage	-map_metadata -1, H.264 CRF18, -movflags +faststart, -pix_fmt yuv420p
5	Injection metadata	title + date depuis plan_de_vol.json. Aucun autre tag.
6	Touch timestamp	Aligne la date systeme du fichier sur la date de production
7	QA post	Verifie codec H.264, absence de tags suspects
8	Rapport HTML	Metriques video, checklist QA, format livre, vitesse appliquee, date

6. FLUX DE PRODUCTION COMPLET — DORN V3 [V3 MAJ]

L'opérateur donne une idée de 10 mots et une durée cible. La machine produit le Short. Les metaprompts s'exécutent dans Gemini via chat manuel.

ETAPE	QUI	ACTION	SORTIE
META_POLUX	Opérateur + Gemini (chat)	Donne sujet + these + PNG + durée cible [V3]	plan_de_vol.json (avec timing calculé)
F01 POLUX	Script Python Colab	Valide JSON + strip EXIF PNG	JSON valide + PNG propres
META_CAMERA	Opérateur + Gemini (chat)	Donne video ref + JSON	camera_plan injecte dans JSON
F02 CASTELLAN	Streamlit Colab	Simule animation (slider sec.), édite, fige	JSON fige (validated_by_magos)
F03A SIGISMUND	Remotion Colab	Build + test 30 frames	Preview 30 frames
F03B SIGISMUND	Modal serverless	Dispatch 3 workers chunks	N chunks parallèles
F03C SIGISMUND	FFmpeg Colab	Merge chunks	video_render.mp4
F04A INWIT	Player HTML Colab	Visionnage + choix vitesse (0.5x → 2x) [V3]	playback_speed valide
F04B INWIT	FFmpeg Colab	Speed filter conditionnel + wipe fingerprint	youtube_short/long.mp4

6.1 — Doctrine d'étancheité (héritée de CRUSADER)

- Chaque fregate est une île : elle cherche UNIQUEMENT dans ses propres dossiers IN/
- JAMAIS de référence directe aux dossiers d'autres fregates
- L'opérateur assure le transit manuel entre fregates (CRS_CUSTOS.py valide les transferts)
- Structure : IN/ → CODEBASE/ → OUT/ par fregate, identique à CRUSADER

6.2 — Budget et contraintes absolues

CONTRAINTE	VALEUR
Budget infrastructure	0.00 EUR
Environnement d'exécution	Google Colab (CPU/GPU gratuit)
Cloud render	Modal serverless (tier gratuit)
API Gemini	Chat manuel uniquement — zero appel API dans le code
FPS cible	60 FPS (Remotion + calculateMetadata dynamique)
Format vertical	1080 x 1920 (9:16) — YouTube Short / TikTok
Format horizontal	1920 x 1080 (16:9) — YouTube Long-form
Durée cible — opérateur	Définie en chat Gemini (META_POLUX) — 20s à plusieurs minutes
Vitesse finale — opérateur	Choisie en F04A — 0.5x à 2x — appliquée par FFmpeg en F04B

7. STRUCTURE DRIVE — DRIVE_DORN/ [V3 MAJ]

```

DRIVE_DORN/
■■■ F01_POLUX/
■   ■■■ CODEBASE/
■   ■   ■■■ DRN_F01.ipynb
■   ■   ■■■ drn_f01_polux.py
■   ■■■ IN/
■   ■   ■■■ plan_de_vol.json ← Produit par META_POLUX (4 inputs V3)
■   ■   ■■■ images/*.png    ← Sources manuellement
■   ■■■ OUT/
■       ■■■ plan_de_vol.json ← JSON valide (avec timing calcule)
■       ■■■ images/*.png    ← EXIF nettoyes
■■■ F02_CASTELLAN/
■   ■■■ CODEBASE/
■   ■   ■■■ DRN_F02.ipynb
■   ■■■ IN/
■   ■   ■■■ plan_de_vol.json ← Depuis F01/OUT/
■   ■   ■■■ images/*.png
■   ■■■ OUT/
■       ■■■ plan_de_vol.json ← Fige (validated_by_magos: true)
■■■ F03_SIGISMUND/
■   ■■■ CODEBASE/
■   ■   ■■■ DRN_F03.ipynb
■   ■   ■■■ remotion.config.js
■   ■   ■■■ package.json
■   ■   ■■■ src/
■   ■       ■■■ Composition.jsx      ← calculateMetadata dynamique [V3]
■   ■       ■■■ MathInterpreter.jsx  ← lit timing + pace_factor [V3]
■   ■       ■■■ CurveTracer.jsx     ← reveal_style easing [V3]
■   ■       ■■■ AssetTracker.jsx
■   ■       ■■■ VirtualCamera.jsx
■   ■■■ IN/
■   ■   ■■■ plan_de_vol.json ← Depuis F02/OUT/ (avec camera_plan)
■   ■   ■■■ images/*.png
■   ■■■ OUT/
■       ■■■ video_render.mp4
■■■ F04_INWIT/
■   ■■■ CODEBASE/
■   ■   ■■■ DRN_F04.ipynb
■   ■   ■■■ drn_f04a_viewer.py ← NOUVEAU V3
■   ■   ■■■ drn_f04b_inwit.py  ← FFmpeg finishing
■   ■■■ IN/
■   ■   ■■■ video_render.mp4 ← Depuis F03/OUT/
■   ■   ■■■ plan_de_vol.json ← Pour lire format + titre + playback_speed
■   ■■■ OUT/
■       ■■■ youtube_short.mp4 OU youtube_long.mp4
■       ■■■ rapport_f04.html
■■■ METAPROMPTS/
■   ■■■ META_POLUX.md ← 4 inputs V3 + calcul step_per_frame
■   ■■■ META_CAMERA.md

```

■■■ SHARED/

■ ■■■ CRS_CUSTOS.py ← Herite de CRUSADER, chemins DORN

■■■ CAMPAIGN_LOG.md

8. NOTES FINALES ET PROCHAINES ETAPES [V3 MAJ]

8.1 — Ce qui reste a creer (ordre prioritaire V3)

PRIORITE	LIVRABLE	DEPEND DE
1	META_POLUX.md — metaprompt Gemini (4 inputs, calcul step_per_frame) [V3]	—
2	META_CAMERA.md — metaprompt camera complet	—
3	drn_f01_polux.py — validation JSON + EXIF strip	META_POLUX.md
4	DRN_F02.ipynb — Streamlit + Canvas JS (slider secondes) [V3]	plan_de_vol.json schema
5	src/ Remotion — 5 composants JSX (MathInterpreter timing, CurveTracer easing) [V3]	plan_de_vol.json schema
6	drn_f04a_viewer.py — Player HTML + selecteur vitesse [V3]	video_render.mp4 de F03
7	drn_f04b_inwit.py — FFmpeg finishing conditionnel [V3]	drn_f04a_viewer.py
8	EXO_GENESIS_DRIVE.py — structure Drive DORN	—

8.2 — Axiomes intouchables

Les elements suivants sont des axiomes structurels de DORN. Le framework ATOM-IC ne s'applique jamais a eux.

- L'architecture en fregates etanches (F01 → F02 → F03 → F04A → F04B)
- Le CMS JSON comme unique source de verite entre fregates
- Le Tesseract (unite de livraison finale) et le Penteract (ecosysteme isole)
- La philosophie Colab-first / budget 0.00 EUR
- Le transit manuel entre fregates + validation CRS_CUSTOS.py
- Le paradigme timing dynamique : la math dicte la duree [V3 — axiome scelle]

8.3 — Compatibilite CRUSADER

CRUSADER et DORN sont deux projets independants. DORN n'est pas un fork de CRUSADER. Technologies portees depuis CRUSADER (cout de debogage deja paye) :

- Modal + chunking parallele 3 workers (17h de debogage validees dans CRUSADER)
- Pattern Colab notebook → Drive → fregate suivante
- calculateMetadata Remotion pour dimensions/duree dynamiques depuis JSON
- Flag --gl swangle Remotion sur Colab (sans ca, rendu casse)
- +faststart FFmpeg (obligatoire YouTube/TikTok)
- CRS_CUSTOS.py — logique de validation des transferts inter-fregates

// FIN DE TRANSMISSION // PENTERACT DORN V3 // TOUTES FREGATES SCELLEES //
VIIe LEGION // 2026-05-29 // IMPERATOR SOMNIUM